Academia\FrontEnd\HTML\Academia_Student.html

```
<!--
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia
November 1, 2026
-->
<!DOCTYPE html>
<html>
<head>
    <!-- UTF-8 so names with accents or punctuation survive the round trip -->
    <meta charset="UTF-8">
    <title>Academia - Student Portal</title>
    <!-- Shared brand stylesheet (navy, royal blue, gold, Poppins) -->
    <link rel="stylesheet" href="../CSS/Academia.css">
</head>

<!-- data-portal "student" is read by the script on page load -->
<body data-portal="student">

    <!-- HEADER BAR -->
    <div class="header">
        <b>Academia Student Portal</b>
        <div>
            <!-- Filled by the script with "Welcome, Alex Rivera". It shows "Loading -->
            <span id="welcome-name">Loading...</span> |
            <!-- Logout clears sessionStorage and returns to the login page -->
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>
    </div>

    <!-- MAIN LAYOUT: SIDEBAR + CONTENT -->
    <div class="main-layout">

        <!-- LEFT SIDEBAR -->
        <div class="sidebar">
            <h3>Menu</h3>
            <!-- showPage() switches which of the two content blocks below is visible -->
            <a href="#" onclick="showPage('dashboard')">Dashboard</a>
            <a href="#" onclick="showPage('courses')">All Courses</a>
            <a href="#" onclick="logout(); return false;">Logout</a>
        </div>

        <!-- CONTENT AREA -->
        <div class="container">

            <!-- The error banner. The script shows it and writes a reason into it when the
fetch fail -->
            <p id="page-error"
```

```
                style="display:none; color:#b00020; font-weight:bold;"></p>

            <!-- VIEW 1: DASHBOARD (visible on load) -->
            <div id="dashboard-page" class="page-content">

                <h2>Student Dashboard</h2>

                <!-- Profile lines. Every "-" below is replaced on load -->
                <p><b>Name:</b> <span id="student-name">-</span></p>
                <p>
                    <b>Student ID:</b> <span id="student-id">-</span> |
                    <b>Grade:</b> <span id="student-grade">-</span>
                </p>
                <p><b>Current GPA:</b> <span id="student-gpa">-</span></p>

                <br>
                <h3>My Grades</h3>
                <!-- The whole table (header row and body rows) is written by the script from studentState -->
                <table border="1" width="100%" id="grades-table"></table>

                <br>
                <h3>Attendance Record</h3>
                <!-- From studentState.attendance; present 1 -> "Present" -->
                <table border="1" width="100%" id="attendance-table"></table>

                <br>
                <h3>Upcoming Assignments</h3>
                <!-- From studentState.assignments, already sorted by due date -->
                <table border="1" width="100%" id="assignments-table"></table>

                <br>
                <h3>Announcements</h3>
                <!-- One card per announcement, built from studentState.announcements (title, message, date) -->
                <div id="announcements"></div>
            </div>

            <!-- VIEW 2: COURSES (hidden on load) -->
            <div id="courses-page" class="page-content" style="display:none;">
                <h2>Enrolled Courses</h2>
                <br>
                <!-- One card per row of studentState -->
                <div id="courses-list"></div>
            </div>

        </div>
    </div>

    <div class="footer"><p>&copy; 2026 Academia Project</p></div>
```

```
96
97      <!-- The shared script. It also contains an optional inline version below, commented out,
    -->
98      <script src="../JS/Academia.js"></script>
99
100 <!-- OPTIONAL: THE SAME LOGIC INLINE (copy after the Academia -->
101 <script>
102 if (typeof loadState !== "function") {
103
104     /* Where the token and the signed-in user are kept. sessionStorage is per
105        tab and is cleared when the tab closes. */
106     var TOKEN_KEY = "academiaToken";
107
108     /* Escapes text before it goes into innerHTML. Without this, a name or an
109        announcement containing < or " could break the page. */
110     function esc(v) {
111         return String(v == null ? "" : v).replace(/[&<>"']/g, function (c) {
112             return { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" }
    [c];
113         });
114     }
115     function $(id) { return document.getElementById(id); }
116     function setHtml(id, html) { var el = $(id); if (el) el.innerHTML = html; }
117     function setText(id, txt) { var el = $(id); if (el) el.innerText = txt; }
118
119     /* Shows the red banner. Falls back to an alert if the element is missing. */
120     function showError(msg) {
121         var el = $("page-error");
122         if (!el) { alert(msg); return; }
123         el.style.display = "block";
124         el.innerText = msg;
125     }
126
127     /* Builds a full <table>: header row, then one row per data row.
128        An empty list produces a single row explaining that there is none. */
129     function table(headers, rows, emptyText) {
130         var h = "<tr>" + headers.map(function (x) { return "<th>" + esc(x) + "</th>";
    }).join("") + "</tr>";
131         if (!rows.length) {
132             return h + "<tr><td colspan='" + headers.length + "'>" + esc(emptyText) + "</td>
    </tr>";
133         }
134         return h + rows.map(function (r) {
135             return "<tr>" + r.map(function (c) { return "<td>" + esc(c) + "</td>";
    }).join("") + "</tr>";
136         }).join("");
137     }
138     function pct(score) { return Number(score).toFixed(1) + "%"; }
139     function presentText(p) { return p ? "Present" : "Absent"; }
140
141     function logout() {
```

```
        sessionStorage.removeItem(TOKEN_KEY);
        sessionStorage.removeItem("academiaUser");
        window.location.href = "Academia_Auth.html";
    }
    function authHeaders(extra) {
        var h = extra || {};
        h["X-Auth-Token"] = sessionStorage.getItem(TOKEN_KEY) || "";
        return h;
    }

    /* Fetches one of the C-generated state files and returns the state object.
       See step 4 and 5 in the header comment: the response is JavaScript, so
       it is read as text and evaluated. */
    function loadState(path, varName) {
        return fetch(path, { headers: authHeaders() }).then(function (res) {
            if (res.status === 401) { logout(); throw new Error("Session expired. Please sign in again."); }
            if (res.status === 403) { throw new Error("This account is not allowed to view that page."); }
            if (!res.ok) { throw new Error("Server error (" + res.status + ")."); }
            return res.text();
        }).then(function (text) {
            var state = new Function(text + "\nreturn " + varName + ";")();
            if (!state || state.success !== true) { throw new Error("The database has no record for this account."); }
            return state;
        });
    }

    /* ---------- login (used by Academia_Auth.html, kept here for safety) ---------- */
    var ROLE_PAGE = { student: "Academia_Student.html", parent: "Academia_Parents.html", staff: "Academia_Staff.html" };

    function goPortal() {
        var u = $("username"), p = $("password"), r = $("role"), err = $("err");
        if (!u || !p || !r) return;
        if (err) err.innerText = "";
        var username = u.value.trim(), password = p.value.trim(), role = r.value;
        if (username === "" || password === "") {
            if (err) err.innerText = "Please enter both username and password.";
            return;
        }
        fetch("/api/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ username: username, password: password, role: role })
        }).then(function (res) { return res.json(); }).then(function (data) {
            if (!data.success) {
                if (err) err.innerText = data.message || "Invalid login credentials.";
                return;
            }
```

```
            sessionStorage.setItem(TOKEN_KEY, data.token);
            sessionStorage.setItem("academiaUser", JSON.stringify(data.user));
            window.location.href = ROLE_PAGE[data.user.role] || "Academia_Auth.html";
        }).catch(function () {
            if (err) err.innerText = "Cannot reach the Academia server. Start it and open http://localhost:3000.";
        });
    }

    /* ---------- student page logic ---------- */
    function showPage(pageId) {
        var d = $("dashboard-page"), c = $("courses-page");
        if (!d || !c) return;
        d.style.display = pageId === "dashboard" ? "block" : "none";
        c.style.display = pageId === "courses" ? "block" : "none";
    }

    function initStudent() {
        loadState("/api/student-data", "studentState").then(function (s) {
            var st = s.student;
            var full = st.firstName + " " + st.lastName;

            /* header + profile */
            setText("welcome-name", "Welcome, " + full);
            setText("student-name", full);
            setText("student-id", st.userId);
            setText("student-grade", st.gradeLevel);
            setText("student-gpa", Number(st.gpa).toFixed(2) + " / 4.00");

            /* tables */
            setHtml("grades-table", table(["Course Title", "Grade"],
                s.grades.map(function (g) { return [g.course, pct(g.score)]; }),
                "No grades recorded yet."));

            setHtml("attendance-table", table(["Course Title", "Status"],
                s.attendance.map(function (a) { return [a.course, presentText(a.present)]; }),
                "No attendance recorded yet."));

            setHtml("assignments-table", table(["Course", "Assignment", "Due"],
                s.assignments.map(function (a) { return [a.course, a.assignmentName, a.dueDate]; }),
                "No assignments."));

            /* course cards */
            setHtml("courses-list", s.courses.length ? s.courses.map(function (c) {
                return "<div class='course-card'><h3>" + esc(c.course) + "</h3><p>" +
                       (c.status ? "Enrolled" : "Not active") + "</p></div><br>";
            }).join("") : "<p>You are not enrolled in any course.</p>");

            /* announcement cards */
```

```
            setHtml("announcements", s.announcements.length ? s.announcements.map(function
(a) {
                return "<div class='course-card'><h3>" + esc(a.title) + "</h3><p>" +
esc(a.message) +
                       "</p><small>" + esc(a.date) + "</small></div><br>";
            }).join("") : "<p>No announcements.</p>");
        }).catch(function (e) { showError(e.message); });
    }

    /* Start: only for a portal page (this one has data-portal="student"). */
    window.addEventListener("load", function () {
        var portal = document.body.getAttribute("data-portal");
        if (portal !== "student") return;
        if (!sessionStorage.getItem(TOKEN_KEY)) { window.location.href =
"Academia_Auth.html"; return; }
        initStudent();
    });
}
</script>
</body>
</html>
```